

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA0706

COURSE NAME: COMPUTER NETWORKS

COURSE OUTCOME COVERED

CO2: *Configure IP addressing schemes, subnetting, and basic routing techniques using simulation tool, for efficient network design and topology. (BL3)*

Assessment Tool Weightage: 40%

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CSA0706 — COMPUTER NETWORKS

Assessment Tool 1 — Analytical Problem Solving: Answers

CO2: Configure IP addressing schemes, subnetting, and basic routing techniques for efficient network design

Q1. CRC Error Detection — Message 11100011, Generator $X^3 + 1$

(i) Polynomial Long Division — Message to Transmit

Generator polynomial $X^3 + 1 \rightarrow$ binary 1001 (degree $r = 3$, so 3 zero bits are appended to the message before division).

Message: 11100011

Message + 3 appended zeros: 11100011000

Divide 11100011000 by 1001 using XOR (modulo-2) long division:

Dividend so far	XOR with Generator (1001)	Result
1110	1001	0111
1110 (bring down 0)	1001	0111
1110 (bring down 0)	1001	0111
1111 (bring down 1)	1001	0110
1101 (bring down 1)	1001	0100
1000 (bring down 0)	1001	0001
0010 (bring down 0) — leading 0, no XOR	—	0010
0100 (bring down 0) — leading 0, no XOR	—	0100

CRC remainder = 100

Transmitted codeword T = message + remainder = 11100011100 $\rightarrow$ 11100011100

Check: dividing 11100011100 by 1001 gives remainder 000, confirming the codeword is correct.

(ii) Leftmost Bit Inverted — Receiver's CRC Check

Transmitted codeword: 11100011100. Inverting the leftmost bit ($1 \rightarrow 0$) gives the received bit stream:

Received: 01100011100

Dividing the received bit stream 01100011100 by the generator 1001 (XOR division) gives a remainder of **010** — a non-zero remainder.

How the receiver detects the error: the receiver always divides the received bit stream by the same generator polynomial the sender used. If the codeword arrives unchanged, this division always leaves a remainder of exactly 000. Here the remainder comes out to 010 (non-zero), which immediately tells the receiver that the bit stream was corrupted in transit — the error is detected even though the receiver has no way of independently knowing what the original message was.

Q2. Throughput and Channel Capacity

(a) Throughput

Bandwidth = 10 Mbps; average 12,000 frames/minute passed, each carrying 10,000 bits on average.

Frames per second = $12,000 \div 60 = 200$ frames/sec

Throughput = frames per second $\times$ bits per frame = $200 \times 10,000 = 2,000,000$ bps

Throughput = 2,000,000 bps = 2 Mbps (only 2 Mbps of the available 10 Mbps bandwidth is actually being used, on average).

(b) Channel Capacity (Shannon's Theorem)

Bandwidth $B = 3000$ Hz (300–3300 Hz); Signal-to-Noise Ratio (SNR) = 3162.

Shannon capacity formula: $C = B \times \log_2(1 + \text{SNR})$

$$C = 3000 \times \log_2(1 + 3162) = 3000 \times \log_2(3163) = 3000 \times 11.627$$

$C \approx 34,881 \text{ bps} \approx 34.88 \text{ kbps}$ — the theoretical maximum error-free data rate this channel can support.

Q3. Hamming (12, 8) Error-Correcting Code — Hospital Monitoring System

Note: The question describes the hospital's scenario and states that a Hamming (12,8) code is used, but the uploaded file does not include a specific instruction (e.g. "encode the following 8-bit reading") or sample data bits after this — the question text ends at "...implements the Hamming (12,8) error-correcting code." The full explanation below covers how and why Hamming(12,8) works, worked through on an illustrative 8-bit reading so every step is demonstrated concretely — substitute the actual patient-data bits if your assignment sheet specifies them.

Why Hamming(12,8) Fits This Scenario

For $m = 8$ data bits, the number of parity bits r must satisfy $2^r \geq m + r + 1$. With $r = 4$: $2^4 = 16 \geq 8 + 4 + 1 = 13$ ✓ ($r = 3$ fails, since $2^3 = 8 < 13$). So 4 parity bits are added to the 8 data bits, giving a 12-bit codeword — the Hamming(12,8) code. Unlike simple CRC (which only detects errors and then requires a retransmission), a Hamming code can locate and correct a single-bit error immediately at the receiver — exactly what the hospital needs, since it cannot afford to wait for a retransmitted heart-rate/blood-pressure reading.

Encoding an Illustrative 8-bit Reading (data = 11100011)

Parity bits are placed at positions that are powers of two (1, 2, 4, 8); data bits fill the remaining positions (3, 5, 6, 7, 9, 10, 11, 12):

Position	1	2	3	4	5	6	7	8	9	10	11	12
Bit	p1	p2	d1=1	p4	d2=1	d3=1	d4=0	p8	d5=0	d6=0	d7=1	d8=1

Each parity bit is the XOR of the data bits at positions whose binary index includes that parity's bit:

- p1 (covers 1,3,5,7,9,11) = $1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 1$
- p2 (covers 2,3,6,7,10,11) = $1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 1 = 1$
- p4 (covers 4,5,6,7,12) = $1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 1$
- p8 (covers 8,9,10,11,12) = $0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 = 0$

Transmitted 12-bit codeword: 1 1 1 1 1 1 0 0 0 1 1 (positions 1–12)

Detecting and Correcting a Single-Bit Error

Suppose electromagnetic interference flips the bit at position 7 during wireless transmission: received codeword = 1 1 1 1 1 1 1 0 0 1 1 1

Received: 1 1 1 1 1 1 1 0 0 1 1 1 (bit 7 flipped from 0 to 1)

The receiver recomputes each parity check on the received bits and adds up the positions of any check that fails:

- Check p1 (1,3,5,7,9,11) fails → contributes 1
- Check p2 (2,3,6,7,10,11) fails → contributes 2
- Check p4 (4,5,6,7,12) fails → contributes 4
- Check p8 (8,9,10,11,12) passes → contributes 0

Syndrome = $1 + 2 + 4 + 0 = 7$ — this directly points to position 7 as the location of the error. The receiver flips bit 7 back and recovers the original, correct codeword — **without needing any retransmission**, satisfying the hospital's real-time requirement.

Q4. Subnetting and IPv6 Addressing

(a) IPv4 Subnetting — 172.16.10.0/24 with a /27 Mask

Bits borrowed for subnetting = $27 - 24 = 3$ bits → Number of subnets = $2^3 = 8$

Host bits remaining = $32 - 27 = 5$ bits → Usable hosts per subnet = $2^5 - 2 = 30$

(2 addresses per subnet — the network address and the broadcast address — are reserved and not usable by hosts.)

Subnet #	Network Address	Usable Host Range	Broadcast Address
1	172.16.10.0/27	.1 – .30	172.16.10.31
2	172.16.10.32/27	.33 – .62	172.16.10.63
3	172.16.10.64/27	.65 – .94	172.16.10.95
4	172.16.10.96/27	.97 – .126	172.16.10.127
5	172.16.10.128/27	.129 – .158	172.16.10.159
6	172.16.10.160/27	.161 – .190	172.16.10.191
7	172.16.10.192/27	.193 – .222	172.16.10.223
8	172.16.10.224/27	.225 – .254	172.16.10.255

172.16.10.78 falls in the range 64–95 → it belongs to subnet **172.16.10.64/27**.

- Network address: 172.16.10.64
- Broadcast address: 172.16.10.95
- Usable host range: 172.16.10.65 – 172.16.10.94

172.16.10.95 — yes, it falls within the same subnet's address range (172.16.10.64 – .95), but it is exactly this subnet's **broadcast address**, not a usable host address; it cannot be assigned to a device.

(b) IPv6 Addressing — 2001:0db8:0000:0000:0000:ff00:0042:8329

Compressed (shortest) form: 2001:db8::ff00:42:8329

(leading zeros dropped in each group: 0db8 → db8, 0042 → 42; the three consecutive all-zero groups are replaced once with "::")

Expanded back to full notation: 2001:0db8:0000:0000:0000:ff00:0042:8329

Network prefix with /64: 2001:0db8:0000:0000::/64 (equivalently 2001:db8::/64) — the first 64 bits (first 4 groups) form the network portion.

Total possible host addresses in this /64 network: $2^{128-64} = 2^{64} = 18,446,744,073,709,551,616$ addresses.

Q5. Online Examination Server — Sender/Receiver Speed Mismatch

8,000 students download exam papers simultaneously from a 10 Gbps server; students on slower connections (Wi-Fi/mobile) report incomplete downloads, retransmissions, delayed ACKs, and portal freezing, while the server keeps transmitting at a high rate.

Primary Networking Problem

This is fundamentally a **flow control problem** — a mismatch between how fast the sender (exam server) transmits data and how fast each receiver (student device) can accept and process it. The server's 10 Gbps capacity lets it push data far faster than a 5–20 Mbps student connection (and that device's own buffer/CPU) can drain it, so the receiver's buffer fills up and overflows.

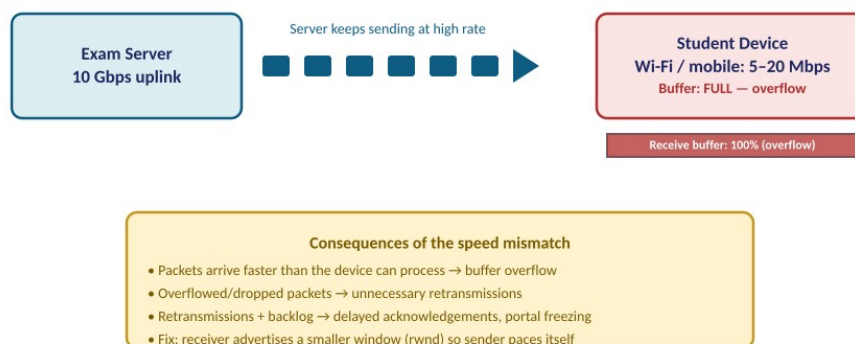


Figure 1 — The exam server transmits far faster than a slow student connection can absorb, filling the receive buffer and causing overflow, drops, and retransmissions.

How the Speed Mismatch Affects Communication

- Receiver buffer overflow: incoming packets arrive faster than the student's device can read them out of its buffer into the application, so once the buffer is full, further arriving packets are simply dropped.
- Unnecessary retransmissions: dropped packets are eventually detected as missing (via timeout or duplicate ACKs) and re-sent, adding extra load to already-congested slower links and wasting the network capacity that is actually available.
- Delayed acknowledgements: an overwhelmed device is slower to process each segment and generate its ACK, so the sender perceives higher round-trip time and reacts more conservatively (or, without proper flow control, keeps overloading it further).
- Portal freezing / incomplete downloads: from the student's perspective, the application layer stalls while the transport layer is stuck re-requesting dropped data, producing exactly the symptoms described — incomplete downloads and a frozen portal.

The Fix — Flow Control

This is precisely the problem transport-layer flow control (e.g., TCP's sliding-window mechanism) is designed to solve: each receiver advertises a receive window (rwnd) reflecting how much buffer space it actually has free. The sender is required to keep its unacknowledged, in-flight data within that advertised window, automatically slowing down for students on slow connections instead of blasting data at the server's full 10 Gbps capacity regardless of what each receiver can handle.

At the scale described here — 8,000 simultaneous slow receivers — this individual flow-control issue also compounds into network-level **congestion** on the shared links leading to those slower connections, so a complete fix combines per-connection flow control (matching sender rate to each receiver's capacity) with congestion control and techniques such as staggered/scheduled distribution of exam papers, so all 8,000 downloads are not forced to compete for the same bottleneck bandwidth at exactly 10:00 AM.